

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет прикладной математики, информатики и механики

Кафедра программного обеспечения
и администрирования информационных систем

**Trie-деревья. Задача о количестве слов,
удовлетворяющих условию отбора**

Курсовая работа

Направление 02.03.03. Математическое обеспечение
и администрирование информационных систем

Зав. кафедрой _____

Обучающийся _____

Руководитель _____

к. ф.-м. н., Барановский Е. С.

2 курс, 9 группа, Зяблова А. А.

д. ф.-м. н., проф. Артёмов М. А.

Воронеж 2026

Аннотация

Данная работа позволяет выявить особенности работы с Trie-деревьями на примере языка C++.

В работе рассмотрены возможности, предоставляемые данным языком для работы с Trie-деревьями, и представлена реализация программы, обеспечивающая построение дерева и решение задач подсчета слов по различным условиям отбора.

Тема данной работы имеет важное учебное значение и позволяет изучить основные принципы и идеи организации Trie-деревьев на примере выбранного языка C++.

Содержание

Введение	5
1. Постановка задачи.....	6
2. Анализ задачи	7
2.1. Общий анализ задачи	7
2.2 Предполагаемые структуры данных	7
2.3. Предполагаемая структура программы.....	7
2.3.1. Предполагаемый план модуля для работы с отдельным узлом дерева	8
2.3.2. Предполагаемый план модуля для работы с Trie-деревом.....	8
2.3.3. Предполагаемая структура главной программы.....	8
3. Средства реализации	10
4. Требования к программному и аппаратному обеспечению	11
5. Интерфейс пользователя	12
6. Реализация	15
6.3. Структура функциональных блоков.....	16
6.4. Описание модулей	17
6.4.1. План модуля для работы с отдельным узлом дерева.....	17
6.4.2. План модуля для работы с Trie-деревом	17
6.4.3. План модуля основной программы	19
7. План тестирования	21
7.1. Тест 1. «Вывод дерева»	21
7.2. Тест 2. «Вывод всех слов дерева»	21
7.3. Тест 3. «Добавление слова в дерево»	21
7.4. Тест 4. «Удаление слова из дерева»	22
7.5. Тест 5. «Загрузка слов из файла».....	22
7.6. Тест 6. «Подсчет всех слов в дереве».....	23
7.7. Тест 7. «Подсчет слов, содержащих заданную букву»	23

7.8. Тест 8. «Подсчет слов, начинающихся на заданный префикс»	23
7.9. Тест 9. «Некорректный ввод данных».....	24
7.10. Тест 10. «Работа с пустым деревом»	24
Заключение	25
Список литературы	26
Приложение. Листинг программы	27

Введение

Данная работа посвящена изучению Trie-деревьев на языке C++.

Trie-дерево - это структура данных, позволяющая хранить множество строк и выполнять операции поиска, вставки и подсчета слов. Каждый узел дерева содержит массив указателей на потомков (по одному на каждую букву алфавита) и флаг, обозначающий конец слова. В отличие от обычных бинарных деревьев поиска, сравнение ключей здесь не используется - переход по дереву определяется значением очередного символа строки.

Цель данной работы - изучить возможности, предоставляемые языком для работы с Trie-деревьями, и реализовать программу, которая будет обеспечивать построение дерева из слов, хранящихся в файле, и подсчет количества слов по различным условиям отбора:

- подсчет общего количества всех слов в дереве;
- подсчет количества слов, содержащих заданную букву;
- подсчет количества слов, начинающихся на заданный префикс.

Для достижения этой цели необходимо решить следующие задачи:

- выбор средств реализации;
- разработка структуры программы;
- реализация программы, демонстрирующей работу с Trie-деревом;
- тестирование работоспособности программы.

Тема работы имеет важное учебное значение. Она позволяет освоить Trie-деревья и использовать полученные знания при решении практических задач, связанных с обработкой текстовой информации.

1. Постановка задачи

Реализовать программу, которая решает задачу о количестве слов, удовлетворяющих условию отбора.

В программе должны быть реализованы следующие функции: построение дерева из файла, печать дерева, подсчет всех слов в дереве, подсчет слов, содержащих заданную букву, подсчет слов, начинающихся на заданный префикс, меню для выбора операции.

Библиотека для работы с Trie-деревом оформляется в виде отдельного модуля. Основная программа осуществляет загрузку данных из файла и взаимодействие с пользователем через меню.

2. Анализ задачи

2.1. Общий анализ задачи

Для решения поставленной задачи необходимо реализовать программу, которая будет обеспечивать базовые средства для работы с Trie-деревом.

Для этого необходимо разработать пакет инструментов, позволяющих выполнять основные функции:

- добавление слова в дерево;
- удаление слова из дерева;
- рекурсивный обход узлов;
- печать всех слов;
- подсчет количества слов по различным критериям отбора;
- очистка памяти.

Необходимо обеспечить корректную работу предоставляемых функций и защиту от некорректного ввода пользователя.

2.2 Предполагаемые структуры данных

Для удобной работы с информацией, хранящейся в Trie-дереве, необходимы вспомогательные структуры данных. Необходим тип, который будет обозначать структуру, являющуюся узлом дерева.

Узел Trie-дерева должен иметь такие поля:

- массив указателей на потомков размером 26 элементов (по количеству букв латинского алфавита).
- флаг конца слова (eow) - логический тип, показывающий, завершается ли в данном узле какое-либо слово.
- счетчик непустых указателей (countNonEmptyPtr) - целое число, хранящее количество ненулевых указателей в массиве потомков.

2.3. Предполагаемая структура программы

Для удобства в проекте можно будет выделить несколько модулей, помимо основной программы, каждый из которых будет содержать

подпрограммы, предназначенные для выполнения определённой группы действий.

2.3.1. Предполагаемый план модуля для работы с отдельным узлом дерева

Один из модулей должен содержать определение структуры узла.

1. Определение структуры NODE, содержащее основные поля: массив указателей на потомков, флаг конца слова, счетчик непустых указателей.
2. Конструктор по умолчанию, инициализирующий поля нулевыми значениями.
3. Конструктор с параметрами, принимающий значения флага и счетчика.
4. Определение типа указателя на узел (ptrNODE).

2.3.2. Предполагаемый план модуля для работы с Trie-деревом

Данный модуль должен содержать класс Trie-дерева и вспомогательные функции для работы с ним.

1. Класс, содержащий корень дерева и методы для работы с ним: приватные методы для рекурсивного добавления и удаления слов, публичные методы для выполнения основных операций с деревом.
2. Функция для рекурсивного обхода дерева и вывода всех слов на экран.
3. Функция для вывода только тех слов, выводятся только слова, узел которых не имеет потомков.
4. Функция для рекурсивного освобождения памяти, занятой деревом.

2.3.3. Предполагаемая структура главной программы

В главной программе будет реализовано меню, в котором будет осуществлён вызов всех необходимых подпрограмм и координация их совместной работы.

1. Функция, осуществляющая ввод одной строчной латинской буквы с клавиатуры и проверку корректности введенных данных. При некорректном вводе выводит сообщение об ошибке и ожидает повторного ввода.
2. Функция, осуществляющая ввод слова или префикса с клавиатуры и проверку корректности введенных данных (строка должна состоять только из строчных латинских букв). При некорректном вводе выводит сообщение об ошибке и ожидает повторного ввода.
3. Функция, которая проверяет, содержится ли заданное слово в Trie-дереве.
4. Функция, которая подсчитывает общее количество слов в дереве (или поддереве).
5. Функция, которая подсчитывает количество слов, содержащих заданную букву.
6. Функция, которая подсчитывает количество слов, начинающихся на заданный префикс.
7. Функция, которая выводит на экран список доступных операций (от 1 до 9), ожидает ввода номера команды от пользователя и проверяет его корректность. При некорректном вводе выводит сообщение об ошибке и ожидает повторного ввода.
8. Функция, которая запускает меню выбора действий, выполняет выбранное действие и перезапускает меню до тех пор, пока пользователь не выберет пункт 9 («Выход»). Перед выполнением действий 1, 2, 4, 6, 7, 8 проверяет, не пусто ли дерево; если дерево пусто, выводит сообщение об ошибке и возвращает в меню.
9. Функция, являющаяся точкой входа в программу.

3. Средства реализации

В качестве языка программирования был выбран язык C++. Возможности данного языка предоставляют все необходимые инструменты для реализации поставленных в данной работе целей и задач. При реализации проекта использовалась среда разработки Microsoft Visual Studio 2022.

4. Требования к программному и аппаратному обеспечению

Приложение предназначено для использования на компьютерах, с установленной операционной системой Windows 10 и выше.

Требования к аппаратному обеспечению соответствуют минимальным требованиям операционной системы.

5. Интерфейс пользователя

В начале работы программы перед пользователем появляется меню выбора действий (рис. 5.1).

```

----- МЕНЮ -----
1. Напечатать дерево
2. Напечатать слова дерева
3. Добавить слово
4. Удалить слово
5. Заполнить дерево из файла
6. Посчитать все слова
7. Посчитать слова, содержащие заданную букву
8. Посчитать слова, начинающиеся на заданный префикс
9. Выход
-----
Выберите действие ->

```

Рис. 5.1 Окно программы при запуске

Пользователь осуществляет вызов необходимой функции, отвечающей за определенное действие путём ввода соответствующей цифры. В процессе выполнения задачи программа выводит результате работы функции либо сообщение о выполнении операции. Примеры работы программы при выборе различных действий (рис. 5.2 – 5.4).

```

----- МЕНЮ -----
1. Напечатать дерево
2. Напечатать слова дерева
3. Добавить слово
4. Удалить слово
5. Заполнить дерево из файла
6. Посчитать все слова
7. Посчитать слова, содержащие заданную букву
8. Посчитать слова, начинающиеся на заданный префикс
9. Выход
-----
Выберите действие -> 5
Введите имя файла для чтения данных (например, input.txt): data.txt
Файл успешно прочитан.
Добавлено слов: 7
Пропущено некорректных слов (с цифрами, кириллицей или спецсимволами): 4

```

Рис. 5.2 Заполнение дерева из файла

```

----- МЕНЮ -----
1. Напечатать дерево
2. Напечатать слова дерева
3. Добавить слово
4. Удалить слово
5. Заполнить дерево из файла
6. Посчитать все слова
7. Посчитать слова, содержащие заданную букву
8. Посчитать слова, начинающиеся на заданный префикс
9. Выход
-----
Выберите действие -> 2

--- Слова дерева ---
apple
apricot
banana
cart
cat
tree
zebra

```

Рис. 5.3 Печать слов дерева

```

----- МЕНЮ -----
1. Напечатать дерево
2. Напечатать слова дерева
3. Добавить слово
4. Удалить слово
5. Заполнить дерево из файла
6. Посчитать все слова
7. Посчитать слова, содержащие заданную букву
8. Посчитать слова, начинающиеся на заданный префикс
9. Выход
-----
Выберите действие -> 3
Введите слово для добавления: cat
Слово cat успешно добавлено.

```

Рис. 5.4 Ввод нового слова

Если при выборе действия допущена ошибка, то на экран будет выведено соответствующее сообщение и пользователь вернется к выбору действия (рис. 5.5)

```

----- МЕНЮ -----
1. Напечатать дерево
2. Напечатать слова дерева
3. Добавить слово
4. Удалить слово
5. Заполнить дерево из файла
6. Посчитать все слова
7. Посчитать слова, содержащие заданную букву
8. Посчитать слова, начинающиеся на заданный префикс
9. Выход
-----
Выберите действие -> g
Ошибка ввода! Введите число от 1 до 9.
Выберите действие -> 0
Ошибка ввода! Введите число от 1 до 9.
Выберите действие ->

```

Рис. 5.5 Ошибка ввода пункта меню

Если при вводе слова допущена ошибка, то на экран будет выведено соответствующее сообщение и пользователь вернется к вводу (рис. 5.6)

```

----- МЕНЮ -----
1. Напечатать дерево
2. Напечатать слова дерева
3. Добавить слово
4. Удалить слово
5. Заполнить дерево из файла
6. Посчитать все слова
7. Посчитать слова, содержащие заданную букву
8. Посчитать слова, начинающиеся на заданный префикс
9. Выход
-----
Выберите действие -> 3
Введите слово для добавления: 5cat
Ошибка! Слово/префикс должно состоять только из строчных латинских букв (a-z).
Введите слово для добавления:

```

Рис. 5.6 Ошибка ввода слова

6. Реализация

6.1. Структуры данных

Константы:

```
const size_t ALPHABET_SIZE = 26;
```

Структура узла, из которых состоит Trie-дерево:

```
struct NODE
{
    int countNonEmptyPtr{};
    NODE* ptrs[ALPHABET_SIZE]{};
    bool eow{ false };
    NODE()
    {
        eow = false;
        for (size_t i{}; i < ALPHABET_SIZE; ++i)
            ptrs[i] = nullptr;
    }
    NODE(bool eow, int cnt) : eow(eow), countNonEmptyPtr(cnt)
    {
        for (size_t i{}; i < ALPHABET_SIZE; ++i)
            ptrs[i] = nullptr;
    };
};
```

Класс, предназначенный для работы с деревом:

```
using ptrNODE = NODE*;
void print_word_rec(ptrNODE t, const std::string& word);
void print_vine_rec(ptrNODE t, const std::string& word);
void clear_rec(ptrNODE& t);
class TrieTree
{
private:
    ptrNODE root{};
    void add_rec(ptrNODE& ptr, const std::string& word, size_t i);
    void del_rec(ptrNODE& ptr, const std::string& word, size_t i);

public:
    TrieTree() { root = nullptr; };
    TrieTree(const std::string& filename);
    ~TrieTree() { clear_rec(root); };
    ptrNODE get_root() { return root; };
    void set_root(ptrNODE t) { root = t; }
    void add(const std::string& word);
    void del(const std::string& word);
    void print(bool vine = true);
    bool empty() { return root == nullptr; };};
```

6.2. Взаимодействие функциональных блоков

Проект состоит из двух модулей и основной программы. В каждом из модулей расположены подпрограммы, отвечающие за выполнение определённой группы действий.

Схема взаимодействия функциональных блоков представлена на рис. 6.2.1.

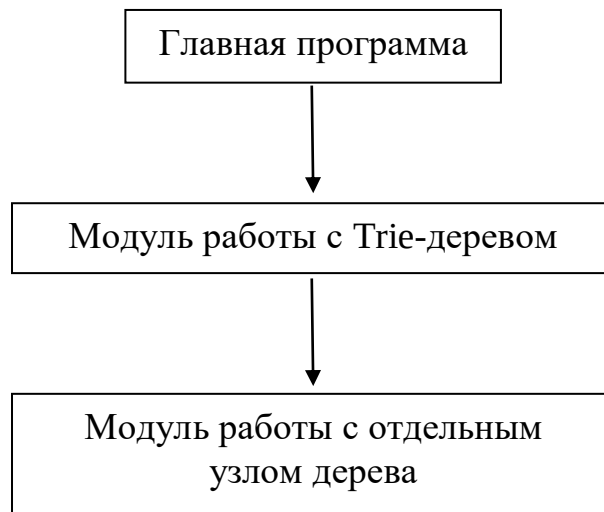


Рис. 6.2.1 Взаимодействие функциональных блоков

6.3. Структура функциональных блоков

В модуле для работы с отдельным узлом дерева расположена структура NODE, представляющая узел дерева.

В модуле для работы с Trie-деревом расположены класс TrieTree и вспомогательные функции, отвечающие за выполнение действий с деревом. Класс TrieTree содержит корень дерева и предоставляет методы для добавления, удаления и печати слов, а также проверки дерева на пустоту.

В основной программе реализованы загрузка данных из файла, меню, функции проверки ввода, функции подсчета слов по различным условиям отбора и функция проверки наличия слова в дереве. Из основной программы происходит вызов методов класса TrieTree.

6.4. Описание модулей

6.4.1. План модуля для работы с отдельным узлом дерева

- `NODE ptrs[ALPHABET_SIZE]*` - массив указателей на потомков. Каждый индекс соответствует букве латинского алфавита. Если указатель равен `nullptr`, то потомок с данной буквой отсутствует.

- `bool eow` - флаг конца слова. Принимает значение `true`, если в данном узле заканчивается какое-либо слово, хранящееся в дереве.

- `int countNonEmptyPtr` - счетчик непустых указателей. Хранит количество ненулевых указателей в массиве `ptrs`. Данный счетчик используется для определения того, является ли узел листом (если счетчик равен нулю, узел не имеет потомков), а также для управления удалением узлов при очистке дерева и удалении отдельных слов.

- `NODE()`

Конструктор по умолчанию. Создает новый узел дерева: устанавливает флаг конца слова `eow` в `false`, счетчик `countNonEmptyPtr` в 0 и обнуляет все указатели массива `ptrs`.

- `NODE(bool eow, int cnt)`

Инициализирующий конструктор. При передаче конкретных значений заполняет поля структуры: устанавливает флаг конца слова `eow` и счетчик непустых указателей `countNonEmptyPtr` в переданные значения. Все указатели массива `ptrs` обнуляются.

6.4.2. План модуля для работы с Trie-деревом

- `TrieTree()`

Конструктор по умолчанию. Инициализирует корень дерева значением `nullptr`.

- `TrieTree(const std::string& filename)`

Инициализирующий конструктор. При передаче имени файла открывает его, считывает слова и добавляет их в дерево вызовом метода `add`.

- `void add_rec(ptrNODE& t, const std::string& word, size_t i)`

Функция, предназначенная для рекурсивного добавления слова в дерево.

- `void del_rec(ptrNODE& t, const std::string& word, size_t i)`

Функция, предназначенная для рекурсивного удаления слова из дерева.

- `void add(const std::string& word)`

Функция, предназначенная для добавления слова в дерево. Вызывает `add_rec`, передавая корень, слово и начальный индекс 0.

- `void del(const std::string& word)`

Функция, предназначенная для удаления слова из дерева. Вызывает `del_rec`, передавая корень, слово и начальный индекс 0.

- `void print(bool vine = false)`

Функция, предназначенная для вывода слов, хранящихся в дереве. При `vine = false` вызывается `print_word_rec` и выводятся все слова. При `vine = true` вызывается `print_vine_rec` и выводятся только листья дерева.

- `bool empty()`

Функция, предназначенная для проверки дерева на пустоту. Возвращает `true`, если корень дерева равен `nullptr`.

- `ptrNODE get_root()`

Функция, предназначенная для получения указателя на корень дерева.

- `void set_root(ptrNODE t)`

Функция, предназначенная для установки корня дерева.

- `void print_word_rec(ptrNODE t, const std::string& word)`

Функция, предназначенная для рекурсивного вывода всех слов дерева. Если в узле установлен флаг `eow`, накопленная строка `word` выводится на экран. Затем функция перебирает всех потомков и для каждого непустого указателя рекурсивно вызывает себя, добавляя к `word` соответствующий символ.

- `void print_vine_rec(ptrNODE t, const std::string& word)`

Функция, предназначенная для рекурсивного вывода листьев дерева. Вывод слова происходит, только если узел не имеет потомков (счетчик `countNonEmptyPtr` равен нулю). Данный счетчик позволяет определить, является ли узел листом, не перебирая весь массив указателей.

- `void clear_rec(ptrNODE& t)`

Функция, предназначенная для рекурсивного освобождения памяти, занятой деревом. Обходит всех потомков и вызывает себя для каждого непустого указателя. Затем удаляет узел и присваивает указателю `nullptr`. При удалении потомка счетчик `countNonEmptyPtr` родительского узла уменьшается, что позволяет корректно отслеживать количество оставшихся потомков и в конечном итоге определять, можно ли удалить сам родительский узел.

6.4.3. План модуля основной программы

- `char get_valid_letter(const std::string& word)`

Функция ввода буквы, осуществляет проверку введенных данных. При некорректном вводе выводит сообщение об ошибке и ожидает повторного ввода.

- `std::string get_valid_word(const std::string& word)`

Функция ввода слова или префикса, осуществляет проверку введенных данных. При некорректном вводе выводит сообщение об ошибке и ожидает повторного ввода.

- `bool contains_word(ptrNODE root, const std::string& word)`

Функция проверки наличия слова в дереве. Последовательно проходит по символам слова. Если все символы пройдены, возвращает значение флага `eow` конечного узла.

- `size_t count_all_rec(ptrNODE ptr)`

Функция подсчета общего количества слов в поддереве. Рекурсивно обходит всех потомков и суммирует количество узлов с установленным флагом `eow`. Возвращает количество найденных слов.

- `size_t count_with_letter_rec(ptrNODE ptr, char letter, bool has_letter)`

Функция подсчета слов, содержащих заданную букву. При обходе отслеживает, была ли встречена искомая буква от корня к текущему узлу. Слово учитывается, если в узле установлен флаг `eow` и буква была встречена. Возвращает количество найденных слов.

- `size_t count_with_prefix(ptrNODE root, const std::string& prefix)`

Функция подсчета слов, начинающихся на заданный префикс. Выполняет спуск по символам префикса. Если префикс не найден, возвращает 0. Если префикс найден, вызывает `count_all_rec` для найденного поддерева. Возвращает количество найденных слов.

- `int menu()`

Функция выбора действия. Выводит на экран основные элементы меню и позволяет выбрать действие. Осуществляет проверку введенных данных.

- `void task_menu(TrieTree& tree)`

Функция выполнения действий. Выполняет действие, выбранное в меню, и вновь запускает функцию выбора действия до тех пор, пока пользователь не выберет пункт «Выход». Перед выполнением операций, требующих непустого дерева, проверяет дерево на пустоту.

- `int main()`

Главная функция, содержащая в себе код, осуществляющий работу программы. Создает объект `TrieTree` и запускает функцию `task_menu`.

7. План тестирования

Для проверки корректности работы данной программы необходимо провести определенный набор тестов, каждый из которых будет проверять работоспособность определенной операции.

Тестирование данного приложения заключается в проверке работоспособности всех подпрограмм, вызываемых при работе приложения, а также в проверке корректности обработки всех запросов пользователя, в том числе при некорректном вводе данных.

7.1. Тест 1. «Вывод дерева»

Цель теста: проверить корректность вывода слов, являющихся листьями дерева.

Описание теста и результат: сначала необходимо загрузить слова из файла (пункт 5) или добавить несколько слов вручную (пункт 3). Выбрав пункт меню «Напечатать дерево», на экран будут выведены слова, конечный узел которых не имеет потомков. Проверить правильность выведенных данных, сверив их с ожидаемым списком листьев.

7.2. Тест 2. «Вывод всех слов дерева»

Цель теста: проверить корректность вывода всех слов, хранящихся в дереве.

Описание теста и результат: сначала необходимо загрузить слова из файла (пункт 5) или добавить несколько слов вручную (пункт 3). Далее выбрать пункт меню «Напечатать слова дерева». На экран должны быть выведены все слова, хранящиеся в дереве. Сверить выведенные слова с ожидаемым содержанием.

7.3. Тест 3. «Добавление слова в дерево»

Цель теста: проверить корректность добавления слова в дерево.

Описание теста и результат: сначала необходимо вывести текущее содержимое дерева (пункт 2). Далее, выбрав пункт меню «Добавить слово», ввести новое слово, состоящее из строчных латинских букв. После ввода слова на экран будет выведено сообщение об успешном добавлении. После необходимо выбрать пункт меню «Напечатать слова дерева» и проверить, что новое слово появилось среди выведенных. При попытке добавить слово, уже существующее в дереве, программа должна вывести сообщение о том, что повторное добавление не требуется.

7.4. Тест 4. «Удаление слова из дерева»

Цель теста: проверить корректность удаления слова из дерева.

Описание теста и результат: сначала необходимо вывести текущее содержимое дерева (пункт 2). Далее, выбрав пункт меню «Удалить слово», ввести удаляемое слово. В случае если такое слово отсутствует в дереве, будет выведено сообщение об ошибке. После успешного удаления будет выведено сообщение об успешном удалении. После завершения работы необходимо выбрать пункт меню «Напечатать слова дерева» и проверить, что удаленное слово исчезло.

7.5. Тест 5. «Загрузка слов из файла»

Цель теста: проверить корректность заполнения дерева из файла.

Описание теста и результат: выбрать пункт меню «Заполнить дерево из файла», ввести имя существующего файла, содержащего слова. Программа должна вывести количество успешно добавленных слов и количество пропущенных некорректных слов. После загрузки выбрать пункт меню «Напечатать слова дерева» и сверить выведенные слова с содержимым файла. При попытке открыть несуществующий файл программа должна вывести сообщение об ошибке.

7.6. Тест 6. «Подсчет всех слов в дереве»

Цель теста: проверить правильность подсчета общего количества слов в дереве.

Описание теста и результат: сначала необходимо загрузить слова из файла, количество которых нам известно. Выбрав пункт меню «Посчитать все слова», на экран будет выведено число. Сверить полученное число с ожидаемым количеством слов в файле.

7.7. Тест 7. «Подсчет слов, содержащих заданную букву»

Цель теста: проверить правильность подсчета слов, содержащих заданную букву.

Описание теста и результат: сначала необходимо загрузить слова из файла, содержимое которого нам известно. Выбрав пункт меню «Посчитать слова, содержащие заданную букву», ввести одну строчную латинскую букву. На экран будет выведено количество слов, в которых эта буква встречается хотя бы один раз. Сверить полученное число с ожидаемым, подсчитанным вручную по содержимому файла. Проверить также случай, когда ни одно слово не содержит заданную букву (результат должен быть равен нулю).

7.8. Тест 8. «Подсчет слов, начинающихся на заданный префикс»

Цель теста: проверить правильность подсчета слов, начинающихся на заданный префикс.

Описание теста и результат: сначала необходимо загрузить слова из файла, содержимое которого нам известно. Выбрав пункт меню «Посчитать слова, начинающихся на заданный префикс», ввести префикс из строчных латинских букв. На экран будет выведено количество слов, начинающихся с указанной последовательности символов. Сверить полученное число с ожидаемым, подсчитанным вручную по содержимому файла. Проверить

также случай, когда ни одно слово не начинается с заданного префикса (результат должен быть равен нулю).

7.9. Тест 9. «Некорректный ввод данных»

Цель теста: проверить корректность обработки некорректного ввода пользователя.

Описание теста и результат: при работе с программой необходимо проверить реакцию на следующие некорректные действия:

- ввод числа вне диапазона 1–9 в главном меню - программа должна вывести сообщение об ошибке и ожидать повторного ввода;
- ввод буквы вместо цифры в главном меню - программа должна вывести сообщение об ошибке и ожидать повторного ввода;
- ввод более одной буквы при запросе одной буквы (пункт 7) - программа должна вывести сообщение об ошибке и ожидать повторного ввода;
- ввод заглавных букв, цифр или спецсимволов в слове (пункты 3, 4, 8) - программа должна вывести сообщение об ошибке и ожидать повторного ввода;
- ввод пустой строки в слове (пункты 3, 4, 8) - программа должна вывести сообщение об ошибке и ожидать повторного ввода.

7.10. Тест 10. «Работа с пустым деревом»

Цель теста: проверить корректность работы программы при пустом дереве.

Описание теста и результат: после запуска программы, не добавляя слова и не загружая файл, поочередно выбрать пункты меню 1, 2, 4, 6, 7, 8. Для каждого из этих пунктов программа должна вывести сообщение об ошибке: «Операция отклонена. Дерево пусто!» и предложить сначала добавить элементы через пункт 3 или загрузить их из файла через пункт 5. Пункт 3 (добавление слова) и пункт 5 (загрузка из файла) должны работать корректно при пустом дереве.

Заключение

Данная работа позволила изучить основные принципы и идеи организации Trie-деревьев на примере языка C++.

В результате решения поставленной задачи была создана и протестирована программа, позволяющая выполнять следующие операции с Trie-деревом:

1. Построение дерева из слов, хранящихся в текстовом файле;
2. Вывод всех слов дерева и вывод листьев дерева на экран;
3. Добавление и удаление слов в дереве;
4. Подсчет количества всех слов, хранящихся в дереве;
5. Подсчет количества слов, содержащих заданную букву;
6. Подсчет количества слов, начинающихся на заданный префикс.

Управление программой происходит при помощи меню и предоставляет пользователю возможность работать с Trie-деревом.

Список литературы

1. Методические указания по оформлению курсовых и выпускных квалификационных работ (факультет ПММ) / М. А. Артемов, Т. В. Азарнова, Н. А. Каплиева [и др.]. – Воронеж: Воронежский государственный университет, 2019. – 12 с.
2. Язык программирования C++. Специальное издание. / Б. Страуструп – Москва: Бином, 2011. – 22 с.
3. Язык программирования C++ (C++11). Лекции и упражнения / С. Прата. – Москва: Вильямс, 2012. – 516 с.

Приложение. Листинг программы

Библиотека Node.h

(модуль работы с отдельным узлом дерева)

Node.h

```
#pragma once
namespace ttree
{
    const size_t ALPHABET_SIZE = 26;
    struct NODE
    {
        NODE* ptrs[ALPHABET_SIZE]{};
        bool eow{};
        int countNonEmptyPtr{};
        NODE()
        {
            eow = false;
            for (size_t i{}; i < ALPHABET_SIZE; ++i)
                ptrs[i] = nullptr;
        }
        NODE(bool eow, int cnt) : eow(eow), countNonEmptyPtr(cnt)
        {
            for (size_t i{}; i < ALPHABET_SIZE; ++i)
                ptrs[i] = nullptr;
        }
    };
}
```

Библиотеки TrieTree.h и TrieTree.cpp

(модуль работы с Trie-деревом)

TrieTree.h

```
#pragma once
#include <iostream>
#include <string>
#include <fstream>

namespace ttree {
    using ptrNODE = NODE*;
    void print_word_rec(ptrNODE t, const std::string& word);
    void print_vine_rec(ptrNODE t, const std::string& word);
    void clear_rec(ptrNODE& t);

    struct TrieTree {
    private:
        ptrNODE root;
        void add_rec(ptrNODE& t, const std::string& word, size_t i);
        void del_rec(ptrNODE& t, const std::string& word, size_t i);
    public:
        TrieTree();
        TrieTree(const std::string& filename);
        ptrNODE get_root() { return root; }
    };
}
```

```

        void set_root(ptrNODE t) { root = t; }
        ~TrieTree();
        void add(const std::string& word);
        void del(const std::string& word);
        void print(bool vine = false);
        bool empty() { return root == nullptr; }

};

}

TrieTree.cpp

#include "TrieTree.h"

void ttree::TrieTree::add_rec(ptrNODE& t, const std::string& word, size_t i)
{
    if (!t)
        t = new NODE;
    if (word.length() - 1 < i)
        t->eow = true;
    else
    {
        if (!t->ptrs[word[i] - 'a'])
            ++t->countNonEmptyPtr;
        add_rec(t->ptrs[word[i] - 'a'], word, i + 1);
    }
}

void ttree::TrieTree::del_rec(ptrNODE& t, const std::string& word, size_t i)
{
    if (t)
    {
        if (i + 1 <= word.length())
        {
            del_rec(t->ptrs[word[i] - 'a'], word, i + 1);
            if (!t->ptrs[word[i] - 'a'])
                --t->countNonEmptyPtr;
        }
        else
        {
            t->eow = false;
            if (t->countNonEmptyPtr == 0)
            {
                delete t;
                t = nullptr;
            }
        }
    }
}

ttree::TrieTree::TrieTree()
{
    root = nullptr;
}

ttree::TrieTree::TrieTree(const std::string& filename)
{
    root = nullptr;
    std::fstream file(filename);
    std::string word{};
    while (file >> word) {
        add(word);
    }
}

```

```

        file.close();
    }

    ttree::TrieTree::~TrieTree()
    {
        clear_rec(root);
    }

    void ttree::TrieTree::add(const std::string& word)
    {
        add_rec(root, word, 0);
    }

    void ttree::TrieTree::del(const std::string& word)
    {
        del_rec(root, word, 0);
    }

    void ttree::TrieTree::print(bool vine)
    {
        if (!vine) {
            print_word_rec(root, "");
        }
        else {
            print_vine_rec(root, "");
        }
    }

    void ttree::print_word_rec(ptrNODE t, const std::string& word)
    {
        if (t) {
            if (t->eow)
                std::cout << word << '\n';
            for (size_t i{}; i < ALPHABET_SIZE; ++i)
                if (t->ptrs[i])
                    print_word_rec(t->ptrs[i], word + char(i + 'a'));
        }
    }

    void ttree::print_vine_rec(ptrNODE t, const std::string& word)
    {
        if (t) {
            if (t->countNonEmptyPtr == 0)
                std::cout << word << '\n';
            for (size_t i{}; i < ALPHABET_SIZE; ++i)
                if (t->ptrs[i])
                    print_vine_rec(t->ptrs[i], word + char(i + 'a'));
        }
    }

    void ttree::clear_rec(ptrNODE& t)
    {
        if (t) {
            for (size_t i{}; i < ALPHABET_SIZE; ++i)
                if (t->ptrs[i]) {
                    --t->countNonEmptyPtr;
                    clear_rec(t->ptrs[i]);
                }
            delete t;
            t = nullptr;
        }
    }

```

Sourse.cpp (модуль работы с основной программой)

```

#include "TrieTree2.h"
#include <functional>
#include <limits>
#include <fstream>

char get_valid_letter(const std::string& word)
{
    std::string input;
    bool isValid = false;

    while (!isValid)
    {
        std::cout << word;
        std::cin >> input;

        if (input.length() == 1 && input[0] >= 'a' && input[0] <= 'z')
        {
            isValid = true;
        }
        else
        {
            std::cout << "Ошибка! Введите одну строчную латинскую букву
(a-z).\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        }
    }
    return input[0];
}

std::string get_valid_word(const std::string& word)
{
    std::string res;
    bool isValid = false;

    while (!isValid)
    {
        std::cout << word;
        std::cin >> res;

        isValid = true;
        for (char ch : res)
        {
            if (ch < 'a' || ch > 'z')
            {
                isValid = false;
                break;
            }
        }

        if (!isValid || res.empty())
        {
            std::cout << "Ошибка! Слово/префикс должно состоять только
из строчных латинских букв (a-z).\n";

```

```

        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        isValid = false;
    }
}
return res;
}

bool contains_word(ttree::ptrNODE root, const std::string& word)
{
    if (!root || word.empty()) return false;

    ttree::ptrNODE curr = root;
    for (char ch : word)
    {
        size_t idx = ch - 'a';
        if (!curr->ptrs[idx])
        {
            return false;
        }
        curr = curr->ptrs[idx];
    }
    return curr->eow;
}

size_t count_all_rec(ttree::ptrNODE ptr)
{
    if (!ptr) return 0;
    size_t count = ptr->eow ? 1 : 0;
    for (size_t i = 0; i < ttree::ALPHABET_SIZE; ++i)
    {
        if (ptr->ptrs[i])
        {
            count += count_all_rec(ptr->ptrs[i]);
        }
    }
    return count;
}

size_t count_with_letter_rec(ttree::ptrNODE ptr, char letter, bool
has_letter)
{
    if (!ptr) return 0;

    size_t count = (ptr->eow && has_letter) ? 1 : 0;

    for (size_t i = 0; i < ttree::ALPHABET_SIZE; ++i)
    {
        if (ptr->ptrs[i])
        {
            char current_char = static_cast<char>(i + 'a');
            bool next_has_letter = has_letter || (current_char ==
letter);
            count += count_with_letter_rec(ptr->ptrs[i], letter,
next_has_letter);
        }
    }
    return count;
}

size_t count_with_prefix(ttree::ptrNODE root, const std::string& prefix)

```

```

{
    if (!root) return 0;
    if (prefix.empty()) return count_all_rec(root);

    ttree::ptrNODE curr = root;
    for (char ch : prefix)
    {
        size_t idx = ch - 'a';
        if (!curr->ptrs[idx])
        {
            return 0;
        }
        curr = curr->ptrs[idx];
    }

    return count_all_rec(curr);
}

int menu()
{
    std::cout << "\n----- МЕНЮ ----- \n";
    std::cout << "1. Напечатать дерево \n";
    std::cout << "2. Напечатать слова дерева \n";
    std::cout << "3. Добавить слово \n";
    std::cout << "4. Удалить слово \n";
    std::cout << "5. Заполнить дерево из файла \n";
    std::cout << "6. Посчитать все слова \n";
    std::cout << "7. Посчитать слова, содержащие заданную букву \n";
    std::cout << "8. Посчитать слова, начинающиеся на заданный префикс \n";
    std::cout << "9. Выход \n";
    std::cout << "----- \n";

    std::function<bool(int)> Lambda = [](int x)->bool {return x >= 1 && x <=
9; };
    int choice{};
    bool isError{ true };
    do
    {
        std::cout << "Выберите действие -> ";
        if (std::cin >> choice && Lambda(choice)) isError = false;
        else
        {
            std::cout << "Ошибка ввода! Введите число от 1 до 9. \n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        }
    } while (isError);

    return choice;
}

void task_menu(ttree::TrieTree& tree)
{
    int choice = menu();

    while (choice != 9)
    {
        if (choice == 1 || choice == 2 || choice == 4 || choice == 6 ||
choice == 7 || choice == 8)
        {
            if (tree.empty())
            {

```



```

std::cout << "\n[ОШИБКА]: Операция отклонена. Дерево
пусто!\n";
std::cout << "Сначала добавьте элементы (пункт 3) или
загрузите их из файла (пункт 5).\n";
choice = menu();
continue;
    }
}

switch (choice)
{
case 1:
{
    std::cout << "\n--- Структура дерева ---\n";
    tree.print(true);
    break;
}
case 2:
{
    std::cout << "\n--- Слова дерева ---\n";
    tree.print(false);
    break;
}
case 3:
{
    std::string word = get_valid_word("Введите слово для
добавления: ");

    if (contains_word(tree.get_root(), word))
    {
        std::cout << "Слово " << word << " уже существует в
дереве (повторное добавление не требуется).\n";
    }
    else
    {
        tree.add(word);
        std::cout << "Слово " << word << " успешно
добавлено.\n";
    }
    break;
}
case 4:
{
    std::string word = get_valid_word("Введите слово для
удаления: ");

    if (contains_word(tree.get_root(), word))
    {
        tree.del(word);
        std::cout << "Слово " << word << " успешно удалено из
дерева.\n";
    }
    else
    {
        std::cout << "[ОШИБКА]: Удаление невозможно. Слово "
<< word << " не найдено в дереве!\n";
    }
    break;
}
case 5:
{
    std::cout << "Введите имя файла для чтения данных (например,
input.txt): ";
    std::string filename;

```

```

std::cin >> filename;

std::ifstream file(filename);
if (!file.is_open())
{
    std::cout << "Ошибка! Не удалось открыть файл " <<
filename << ".\n";
}
else
{
    std::string fileWord;
    size_t addedCount = 0;
    size_t skippedCount = 0;

    while (file >> fileWord)
    {
        bool isValid = true;
        for (char ch : fileWord)
        {
            if (ch < 'a' || ch > 'z')
            {
                isValid = false;
                break;
            }
        }

        if (isValid && !fileWord.empty())
        {
            tree.add(fileWord);
            addedCount++;
        }
        else
        {
            skippedCount++;
        }
    }
    file.close();

    std::cout << "Файл успешно прочитан.\nДобавлено слов:
" << addedCount << "\n";
    if (skippedCount > 0)
    {
        std::cout << "Пропущено некорректных слов (с
цифрами, кириллицей или спецсимволами): " << skippedCount << "\n";
    }
    break;
}
case 6:
{
    size_t totalWords = count_all_rec(tree.get_root());
    std::cout << "Общее количество всех слов в дереве: " <<
totalWords << "\n";
    break;
}
case 7:
{
    char letter = get_valid_letter("Введите букву для проверки:
");
    size_t wordsWithLetter =
count_with_letter_rec(tree.get_root(), letter, false);
    std::cout << "Количество слов, содержащих букву '" << letter
<< "': " << wordsWithLetter << "\n";
    break;
}

```

```

    }
    case 8:
    {
        std::string prefix = get_valid_word("Введите префикс для
поиска: ");
        size_t wordsWithPrefix = count_with_prefix(tree.get_root(),
prefix);
        std::cout << "Количество слов, начинающихся на " << prefix
<< ": " << wordsWithPrefix << "\n";
        break;
    }
    default:
        break;
    }
    choice = menu();
}

int main()
{
    setlocale(LC_ALL, "Russian");

    ttree::TrieTree tree;
    task_menu(tree);

    return 0;
}

```